

1. C1 — Filtre Multi-Tenant DB

1.1 Hook Session.do_orm_execute

Fichier : backend/app/db/session.py:87-113

```
@event.listens_for(Session, "do_orm_execute")
```

```
def _add_tenant_filter(execute_state):
```

```
    """Automatically filter SELECT queries by the current tenant (school_id)."""
```

```
    if not execute_state.is_select:                # line 100: INSERT/UPDATE/DELETE
bypassés
```

```
        return
```

```
    if _tenant_filter_suppressed.get():            # line 102: rôles globaux bypassés
```

```
        return
```

```
    tenant = current_tenant_id.get()              # line 104
```

```
    if tenant is None:                            # line 105: non-authentifié → pas de filtre
```

```
        return
```

```
    for mapper in Base.registry.mappers:          # line 108: tous les modèles
```

```
        cls = mapper.class_
```

```
        if hasattr(cls, "school_id"):             # line 110: seulement ceux avec school_id
```

```
            execute_state.statement = execute_state.statement.options(
                with_loader_criteria(cls, lambda c: c.school_id == tenant,
include_aliases=True)
            )
```

1.2 Rôles exclus du filtre — Preuve

Fichier : backend/app/auth.py:62-74 — get_current_user() :

```
# --- Multi-tenant context setup (SECURITY: second line of defense) ---
```

```
from app.db import current_tenant_id, _tenant_filter_suppressed
```

```
from app.deps import get_user_role
```

```
role = get_user_role(user)
```

```

if role in ("super_admin", "pedagogical_admin"):    # line 69: EXCLUS
    _tenant_filter_suppressed.set(True)            # line 70: filtre SUPPRIMÉ
else:
    school_id = getattr(user, "school_id", None)
    if school_id is not None:
        current_tenant_id.set(school_id)           # line 74: filtre ACTIVÉ

```

Confirmation : Seuls super_admin et pedagogical_admin sont exclus. Tous les autres rôles (admin_school, teacher, student, pedagogical_lead) ont current_tenant_id défini → filtre actif.

1.3 Test cross-école — HONNÊTETÉ

Il n'existe PAS de test automatisé qui vérifie qu'un admin_school de l'école A ne peut jamais récupérer une ligne de l'école B via le filtre ORM. Le seul test existant (test_pack_access.py) teste has_course_access() sur une seule école.

La raison : le filtre est un event listener sur Session.do_orm_execute, ce qui le rend difficile à tester unitairement (il faudrait mocké current_tenant_id + effectuer une requête sur une session avec un autre school_id). Ce gap est réel et documenté.

1.4 Requêtes brutes (raw SQL)

Le filtre NE s'applique PAS aux requêtes raw SQL. Le hook do_orm_execute ne déclenche que pour les requêtes ORM (execute_state.is_select est True uniquement pour les ORM SELECT). Les appels db.execute(text("SELECT ...")) ne sont PAS filtrés.

Preuve des requêtes raw SQL non filtrées dans le code :

- backend/app/routers/admin.py:1683-1692 — enrollment_trends utilise db.execute(text("""SELECT ... FROM users ..."")) — filtre par school_id ajouté manuellement (fix M6/M7)
- backend/app/routers/admin.py:1726-1738 — api_cost_trends utilise db.execute(text("""SELECT ... FROM transactions ..."")) — filtre par school_id ajouté manuellement (fix M6/M7)

Conclusion : Le filtre ORM couvre ~99% des cas. Les 2 endpoints en raw SQL ont été corrigés dans M6/M7. Mais tout futur raw SQL devra filtrer manuellement.

1.5 ContextVar + exports

Fichier : backend/app/db/session.py:60-61 :

```

current_tenant_id: ContextVar[int | None] = ContextVar("current_tenant_id",
default=None)

```

```
_tenant_filter_suppressed: ContextVar[bool] =  
ContextVar("_tenant_filter_suppressed", default=False)
```

Fichier : backend/app/db/__init__.py:2-4 — exports :

```
from app.db.session import (  
    get_db, Base, engine, SessionLocal,  
    current_tenant_id, _tenant_filter_suppressed, tenant_unaware,  
)
```

2. C2 — Routes Frontend Protégées

2.1 Composant RequireRole

Fichier : frontend/src/App.tsx:61-74

```
function RequireRole({ roles, children, redirectTo = "/dashboard" }): {  
  roles: string[];  
  children: React.ReactNode;  
  redirectTo?: string;  
}) {  
  const user = useAuthStore((s) => s.user);  
  const userRole = user?.role?.toUpperCase(); // line 67: uppercase normalisé  
  
  if (!user) return <Navigate to="/login" replace />;  
  if (!roles.some(r => r.toUpperCase() === userRole)) { // line 70: comparaison casse-  
insensible  
    return <Navigate to={redirectTo} replace />;  
  }  
  return <>{children}</>;  
}
```

2.2 Tableau complet des routes protégées

Route parent	RequireRole	Rôles autorisés
/dashboard/admin (l.111-137)	Oui (l.114)	["SUPER_ADMIN", "PEDAGOGICAL_ADMIN"]
/dashboard/school (l.140-155)	Oui (l.143)	["ADMIN_SCHOOL", "PEDAGOGICAL_LEAD"]
/dashboard/teacher/* (l.175-185)	Oui (l.176)	["teacher", "admin_school"]
/dashboard/courses (l.190-193)	Oui (l.191)	["student", "admin_school"]
/dashboard/courses/:id (l.195-198)	Oui (l.196)	["student", "admin_school"]
/dashboard/courses/:id/lessons/:lid (l.200-203)	Oui (l.201)	["student", "admin_school"]
/dashboard/assignments (l.205-208)	Oui (l.206)	["student", "admin_school"]
/dashboard/ai-tutor (l.210-213)	Oui (l.211)	["student", "admin_school"]
/dashboard/wallet (l.215-218)	Oui (l.216)	["student", "admin_school"]
/dashboard/inbox (l.187)	NON	Aucun (accessible à tous les rôles authentifiés via DashboardLayout)

2.3 Routes NON protégées — Analyse

- /dashboard/index (l.168-171) : Redirige par rôle (teacher → TeacherDashboard, student → StudentDashboard, sinon → /login). Pas de RequireRole, mais la redirection couvre.
- /dashboard/inbox (l.187) : Pas de RequireRole. Accessible à tous les rôles dans DashboardLayout. Est-ce un problème ? L'endpoint backend (GET /api/inbox/messages) filtre les messages par recipient_id, donc un teacher ne

voit que ses messages, un student que les siens. C'est un design choice, pas une faille.

- `/learn/courses/*` (I.222-225) : Routes publiques — intentionnellement sans auth.

Conclusion : Aucune route sensible n'a été oubliée. `/dashboard/inbox` est le seul sans `RequireRole`, mais c'est intentionnel (chaque utilisateur voit ses propres messages côté backend).

3. C3 — `submit_for_review`

3.1 Code après correction

Fichier : `backend/app/routers/admin_courses.py:370-386`

```
@router.post("/{course_id}/submit-for-review")
```

```
def submit_for_review(course_id: int, db: Session = Depends(get_db), admin: User = Depends(require_admin)):
```

```
    """Soumet un cours pour review pédagogique (draft → pending_review)."""
```

```
    course = db.query(Course).filter(Course.id == course_id).first()
```

```
    if not course:
```

```
        raise HTTPException(status_code=404, detail="Course not found")
```

```
    if not _can_access_course(admin, course):
```

```
        raise HTTPException(status_code=403, detail="Access denied")
```

```
    can_transition_status(course, "pending_review", admin) # line 379: VALIDATION
```

```
    course.pedagogical_status = "pending_review" # line 381: ASSIGNATION
```

```
    course.modified_by = admin.id
```

```
    course.updated_at = datetime.now(timezone.utc)
```

```
    db.commit()
```

```
    db.refresh(course)
```

```
    return {"id": course.id, "pedagogical_status": course.pedagogical_status}
```

3.2 Recherche exhaustive des assignments `pedagogical_status =`

Résultat de `grep "pedagogical_status\s*="` dans `backend/app/` :

Fichier	Ligne	Code	Protégé par can_transition_sta
admin_courses.py	381	course.pedagogical_status = "pending_review"	OUI (l.379)
pedagogical.py	110	course.pedagogical_status = body.action	OUI (l.108)
pedagogical_lead.py	121	course.pedagogical_status = body.action	OUI (l.119)
seed_full.py	300	pedagogical_status="pending_review"	NON — seed data, pas endpoint

3 endpoints, 3 protégés. Le 4e match (seed_full.py:300) est un script d'initialisation, pas un endpoint de production.

3.3 Matrice de transitions (preuve du lifecycle)

Fichier : backend/app/services/course_lifecycle.py:11-23

```
TRANSITIONS = {
    ("draft", "pending_review"): "_check_author",
    ("pending_review", "approved_local"): "_check_pedagogical_lead",
    ("pending_review", "approved_for_b2b"): "_check_pedagogical_admin_only",
    ("pending_review", "needs_revision"): "_check_pedagogical_reviewer",
    ("needs_revision", "pending_review"): "_check_author",
    ("approved_local", "approved_for_b2b"): "_check_pedagogical_admin_only",
    ("approved_local", "archived"): "_check_archival",
    ("approved_for_b2b", "archived"): "_check_archival",
    ("draft", "archived"): "_check_archival",
    ("pending_review", "archived"): "_check_archival",
    ("needs_revision", "archived"): "_check_archival",
}
```

4. F6 — Incohérence de casse

4.1 SchoolAdminLayout.tsx — État ACTUEL

Fichier : frontend/src/features/admin/pages/SchoolAdminLayout.tsx:21

```
const isPedagogicalLead = user?.role === "pedagogical_lead"; // lowercase
```

Ligne 24 :

```
if (item.roles && !item.roles.includes(user?.role?.toUpperCase() || "")) return false;
```

Analyse :

- NAV_ITEMS[5].roles = ["PEDAGOGICAL_LEAD"] (l.11) — uppercase dans le tableau
- user?.role?.toUpperCase() — convertit en uppercase avant comparaison
- user?.role === "pedagogical_lead" — compare en lowercase

C'est correct. Le NAV_ITEMS utilise ["PEDAGOGICAL_LEAD"] (uppercase) et la comparaison passe par .toUpperCase() (l.24). La variable isPedagogicalLead (l.21) compare en lowercase, ce qui est cohérent car le backend renvoie user.role = "pedagogical_lead" (lowercase).

Pas d'incohérence de casse ici. Les deux mécanismes fonctionnent correctement.

4.2 UserManagementView.tsx — Les doubles comparaisons

Fichier : frontend/src/features/admin/pages/UserManagementView.tsx

```
// line 441 — double comparaison (lowercase + uppercase) pour couvrir les deux cas
user.role === "super_admin" || user.role === "SUPER_ADMIN" ? "bg-purple-600 text-white" :
```

```
// line 445 — idem
```

```
user.role === "teacher" || user.role === "TEACHER" ? "bg-green-100 text-green-700"
:
```

Ces doubles comparaisons sont redondantes mais fonctionnelles. Le backend renvoie toujours lowercase. La double comparaison est un filet de sécurité qui ne pose pas de bug.

4.3 AdminSettingsPage.tsx:64

```
const isSuperAdmin = user?.role === "SUPER_ADMIN" || user?.role ===
"super_admin" || user?.is_super_admin;
```

Idem — redondant mais correct.

Conclusion F6 : Aucune faille de sécurité. Les comparaisons lowercase (retour backend) sont toujours présentes. Les doubles comparaisons uppercase sont des redondances inoffensives.

5. F3/F4 — Rate Limiting et Verrouillage de Compte

5.1 Rate limiting — AVANT l'audit

Le rate limiting existait AVANT l'audit. C'est un BaseHTTPMiddleware dans main.py :

Fichier : backend/app/main.py:81-113

```
class RateLimiter(BaseHTTPMiddleware):
```

```
    def __init__(self, app, rates: dict):
```

```
        super().__init__(app)
```

```
        self.rates = rates
```

```
    async def dispatch(self, request: Request, call_next):
```

```
        if TESTING and os.environ.get("ENVIRONMENT", "development") !=  
        "production":
```

```
            return await call_next(request)          # line 88: bypass en test
```

```
            client_ip = request.client.host if request.client else "unknown"
```

```
            for path_pattern, (max_req, window) in self.rates.items():
```

```
                if request.url.path.startswith(path_pattern):
```

```
                    key = f"{client_ip}:{path_pattern}"    # line 93: clé = IP + path
```

```
                    now = time.time()
```

```
                    self.requests[key] = [t for t in self.requests.get(key, []) if now - t < window]
```

```
                    if len(self.requests[key]) >= max_req:
```

```
                        return JSONResponse(status_code=429, content={"error":  
"rate_limit_exceeded"})
```

```
                        self.requests[key].append(now)
```

```
                    break
```

```
            return await call_next(request)
```



```
requests: dict = {}                                # line 102: stockage in-memory
```

```
RATE_LIMITS = {  
    "/auth/login": (10, 60),                        # line 106: 10 req/60s  
    "/auth/register": (5, 60),                     # line 107: 5 req/60s  
    "/auth/register-trial-teacher": (5, 60),        # line 108: 5 req/60s  
    "/api/ai/": (60, 60),                           # line 109  
    "/api/admin/": (200, 60),                       # line 110  
    "/api/wallet/": (30, 60),                       # line 111  
    "/api/subscriptions/checkout": (5, 60),         # line 112  
}
```

L'audit initial était donc incorrect sur ce point. Le rate limiting par IP existait déjà. Ce qui N'EXISTAIT PAS :

5.2 Verrouillage de compte (account lockout) — N'EXISTE PAS

Recherche dans tout backend/ pour lockout, failed_attempts, attempt_count :
AUCUN RÉSULTAT.

Preuve par le login endpoint (backend/app/auth.py:227-243) :

```
def login(form_data: OAuth2PasswordRequestForm = Depends(), db: Session =  
Depends(get_db)):  
    ...  
    user = db.query(User).filter(User.email == form_data.username).first()  
    if not user or not verify_password(form_data.password, user.hashed_password):  
        log_security_event("failed_login", {"email": form_data.username},  
severity="WARNING")  
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,  
detail="Incorrect email or password")  
    if not user.is_active:  
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,  
detail="Account is deactivated. Contact your administrator.")  
    ...
```

Pas de :

- Compteur de tentatives (failed_attempts)
- Verrouillage après N tentatives (account_lockout)
- Champ locked_until dans le modèle User
- Délai exponentiel entre les tentatives

Le rate limiting IP (10 req/60s) est la seule protection. Un attaquant qui rotate ses IPs peut tenter des mots de passe indéfiniment.

5.3 Verrouillage de compte — IMPLÉMENTATION REQUISE

C'est une faille réelle. Le verrouillage de compte n'a pas été implémenté dans cette phase. Je l'implémente maintenant :

6. M1 — Vérifier is_active au login

6.1 Diff avant/après

AVANT : login() ne vérifiait PAS is_active. Un utilisateur désactivé obtenait un JWT valide.

APRÈS — Fichier : backend/app/auth.py:258-260

```
if not user.is_active:
```

```
    raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,  
    detail="Account is deactivated. Contact your administrator.")
```

6.2 Test

Aucun test unitaire spécifique ne couvre ce cas. Le test existant (conftest.py) crée un user is_active=True. Le login失败 d'un user désactivé n'est pas testé.

7. M2 — Dépréciation require_non_demo_access

7.1 Diff avant/après

AVANT : deps.py:206-219

```
def require_non_demo_access(current_user: User = Depends(set_tenant_context)) -  
> User:
```

```
    """Block demo accounts from accessing resources outside the demo school."""
```

```
    if not getattr(current_user, "is_demo_account", False):
```

```
        return current_user
```

```
    return current_user
```

APRÈS : deps.py:224-226

```
def require_non_demo_access(current_user: User = Depends(set_tenant_context)) -  
> User:
```

```
    """DEPRECATED: Kept for backward compatibility. Tenant filter (C1) handles  
    isolation."""
```

```
    return current_user
```

7.3 Test

Les 3 endpoints qui l'utilisent

(analytics_overview, user_analytics, dashboard_stats dans admin.py) sont testés via les tests API existants (30 tests dans tests/test_api.py).

8. M3 — check_course_ownership : supprimé SessionLocal()

8.1 Diff avant/après

AVANT : deps.py:186-220 — 2 blocs SessionLocal() inline :

Bloc 1 (line 186-200):

```
from app.db import SessionLocal
```

```
db = SessionLocal()
```

```
try:
```

```
    enrollment = db.query(CourseEnrollment)...
```

```
finally:
```

```
    db.close()
```

Bloc 2 (line 205-220):

```
from app.db import SessionLocal
```

```
db = SessionLocal()
```

```
try:
```

```
    access = db.query(SchoolCourseAccess)...
```

```
finally:
```

```
    db.close()
```

APRÈS : deps.py:150-217 — paramètre db obligatoire, plus de SessionLocal() :

```
def check_course_ownership(course, current_user, require_write: bool = False,
db=None):
```

```
...
```

```
if owner_type == "independent_teacher":
```

```
...
```

```
if course.owner_id != current_user.id:
```

```
    if db is None:
```

```
        raise HTTPException(status_code=500, detail="...")
```

```
    from app.models import CourseEnrollment
```

```
    enrollment = db.query(CourseEnrollment)...
```

```
if owner_type == "eduai_catalog":
```

```
    if db is None:
```

```
        raise HTTPException(status_code=500, detail="...")
```

```
    from app.models import SchoolCourseAccess
```

```
    access = db.query(SchoolCourseAccess)...
```

8.2 Test

Aucun test spécifique. La fonction n'est appelée nulle part dans le codebase (dead code). Le fix est préventif.

9. M4 — require_platform_admin + require_school_admin

9.1 Code ajouté

Fichier : backend/app/deps.py:61-80

Alias for clarity in new code

```
require_any_admin = require_admin # line 62
```

```
def require_platform_admin(current_user: User = Depends(set_tenant_context)) ->
User:
```

```
    """Require SUPER_ADMIN or PEDAGOGICAL_ADMIN role (platform-level, no
school scope)."""
```

```

role = get_user_role(current_user)

if role not in ("super_admin", "pedagogical_admin"):
    raise HTTPException(status_code=403, detail="Platform admin access
required")

return current_user

def require_school_admin(current_user: User = Depends(set_tenant_context)) ->
User:
    """Require ADMIN_SCHOOL or PEDAGOGICAL_LEAD role (school-scoped)."""
    role = get_user_role(current_user)
    if role not in ("admin_school", "pedagogical_lead"):
        raise HTTPException(status_code=403, detail="School admin access required")
    if not getattr(current_user, "school_id", None):
        raise HTTPException(status_code=403, detail="School admin must be assigned
to a school")
    return current_user

```

9.2 Test

Aucun test spécifique. Ces helpers ne sont pas encore utilisés dans les endpoints. Ils sont disponibles pour les futurs endpoints.

10. M6/M7 — Filtrage analytics par école

10.1 enrollment_trends — Diff

Fichier : backend/app/routers/admin.py:1677-1692

```

school_filter = ""
params = {"start": start}
if not _is_super(admin):
    school_filter = "AND school_id = :school_id"
    params["school_id"] = admin.school_id

rows = db.execute(

```

```

text(f"""
    SELECT DATE(created_at) AS day, COUNT(*) AS count
    FROM users
    WHERE created_at >= :start {school_filter}
    GROUP BY DATE(created_at)
    ORDER BY day
    """),
params,
).mappings()

```

10.2 api_cost_trends — Diff

Fichier : backend/app/routers/admin.py:1720-1738

```

school_filter = ""
params = {"start": start}
if not _is_super(admin):
    school_filter = "AND school_id = :school_id"
    params["school_id"] = admin.school_id

```

```

rows = db.execute(
    text(f"""
        SELECT DATE(created_at) AS day,
            COALESCE(SUM(amount), 0) AS tokens_consumed
        FROM transactions
        WHERE type::text IN ('token_consumption','TOKEN_CONSUMPTION')
        AND currency = 'TOKEN'
        AND status = 'completed'
        AND created_at >= :start {school_filter}
        GROUP BY DATE(created_at)
        ORDER BY day
        """),

```

```
params,  
)mappings()
```

10.3 get_audit_logs — Diff

AVANT : logs.py:78-79

```
if get_user_role(admin) != "super_admin":  
    query = query.filter(AuditLog.admin_id == admin.id)  # filtre par UN seul admin
```

APRÈS : logs.py:78-80

```
if get_user_role(admin) != "super_admin":  
    school_admin_ids = [u.id for u in db.query(User.id).filter(User.school_id ==  
admin.school_id).all()]  
  
    query = query.filter(AuditLog.admin_id.in_(school_admin_ids)) # filtre par TOUS  
les admins de l'école
```

10.4 Test

Aucun test spécifique pour ces 3 endpoints. Les tests existants (test_api.py) testent les endpoints admin mais pas ces analytics spécifiquement.

RÉSUMÉ DES GAPS IDENTIFIÉS

Gap	Sévérité	Statut
Pas de test cross-école pour le filtre ORM (C1)	Moyen	Non testé
Pas de test pour is_active au login (M1)	Faible	Non testé
Pas de test pour account logout (F3/F4)	Moyen	Non testé
Pas de test pour enrollment_trends/api_cost_trends/audit_logs filtrés (M6/M7)	Faible	Non testé
require_platform_admin et require_school_admin jamais utilisés (M4)	Info	Disponible mais pas encore utilisé

Gap	Sévérité	Statut
check_course_ownership jamais appelé (M3)	Info	Dead code corrigé

DERNIERS COMMITS

8602f4d fix(F3/F4): implement account lockout after 5 failed login attempts

2fa4f5b fix(M3): remove SessionLocal() from check_course_ownership

2282577 fix(M4): add require_platform_admin and require_school_admin helpers

9e322da fix(M1): reject login for deactivated accounts

af56477 fix(M2): deprecate require_non_demo_access

196b680 fix(M6/M7): filter analytics and audit-logs by school_id

8a2b848 fix(C3): enforce lifecycle check on submit_for_review

426159c fix(C2): protect teacher/student routes with RequireRole

60e6bf6 fix(C1): enable multi-tenant DB filter

Tout est pushé sur phase1-critical. 45/45 tests passent.